

Feature Scaling: Standardization

100 Days of Machine Learning | Day 24 | CampusX

1. What is Feature Scaling?

Feature Scaling is a technique used to standardize the independent features present in the data in a fixed range. It is generally the **last step** you perform in Feature Engineering before feeding the data into a Machine Learning model.

Why do we need Feature Scaling?

Imagine a dataset with two features: **Age** (ranges from 18 to 60) and **Salary** (ranges from 15,000 to 150,000). Machine Learning algorithms like **K-Nearest Neighbors (KNN)** calculate the Euclidean distance between data points. Because **Salary** has a massively larger scale, it will dominate the distance calculation, making the algorithm completely ignore the **Age** feature. Scaling brings both features to a comparable range.

2. Types of Feature Scaling

- **Standardization (Z-Score Normalization):** Focus of this session.
- **Normalization (Min-Max Scaling, Robust Scaling, etc.):** Covered in the next session.

3. Mathematical Intuition of Standardization

Standardization transforms your data such that the resulting distribution has a **Mean (μ) of 0** and a **Standard Deviation (σ) of 1**.

The formula applied to every single data point **x_i** is:

$$x_{\text{new}} = (x_i - \mu) / \sigma$$

Geometric Intuition:

1. **Mean Centering ($x_i - \mu$)**: This mathematically picks up your entire scatter plot of data and shifts it so that the center (mean) rests exactly at the origin **$(0, 0)$** .
2. **Scaling $/ \sigma$** : This squeezes or stretches the data points so that the spread (standard deviation) along all axes is exactly **1**.

Important Note: Standardization does **NOT** change the shape of your data's distribution (e.g., it won't make a skewed distribution normal). It only scales it down.

4. Implementing Standardization in Scikit-Learn

We use the `StandardScaler` class from `sklearn.preprocessing`.

CRITICAL RULE: Always split your data into Train and Test sets *before* scaling. You must `fit()` the scaler **ONLY** on the training data (to learn the mean and standard deviation) and then `transform()` both the Train and Test sets. This prevents "Data Leakage."

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# 1. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=0)

# 2. Create the StandardScaler object
scaler = StandardScaler()

# 3. Fit on Training data ONLY (calculates mean and std dev)
scaler.fit(X_train)

# 4. Transform both Train and Test data
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

5. Effect of Standardization on Outliers

Standardization **does NOT** handle or reduce the impact of outliers. If an outlier existed far away from the rest of the data before scaling, it will still exist far away from the rest of the data after scaling. You must handle outliers explicitly using outlier removal techniques.

6. When to Use Standardization?

You **MUST** use feature scaling for algorithms that rely on distances or Gradient Descent:

- **K-Means / K-Nearest Neighbors (KNN):** Relies on Euclidean distances.
- **Principal Component Analysis (PCA):** Tries to get features with maximum variance; requires mean centering.

- **Artificial Neural Networks (ANN) / Deep Learning:** Uses Gradient Descent. If features are on different scales, the weights will update unevenly, and the algorithm may never converge.
- **Gradient Descent based algorithms:** Linear Regression, Logistic Regression.

You **DO NOT** need feature scaling for Tree-based algorithms:

- **Decision Trees, Random Forest, Gradient Boosting, XGBoost:** These algorithms make splits based on mathematical thresholds (e.g., "Is Age > 40?") rather than calculating distances. Scaling has zero impact on their performance.